BUILD AN AI-POWERED WORKFLOW SYSTEM

Sprintwise

PRD to Sprint Planner

A full-stack workflow that turns a feature idea into a structured product brief, prioritized work, and a reviewable sprint plan. Its transparent rules make the decision layer visible without an external AI dependency.

INPUT	PROCESS	STORAGE	OUTPUT
Feature idea	PRD, stories, tasks	JSON records	Sprints and admin

1. System architecture

The browser form sends structured input to a Node.js HTTP API. The planner validates the request, derives product requirements and candidate work, applies scoring and estimation rules, then stores the plan. The interface renders a readable plan; the admin view aggregates saved records.

Layer	Implementation
Frontend	Responsive HTML, CSS, and browser JavaScript. Structured intake, output cards, history, Markdown export, admin dashboard.
Backend	Node.js built-in HTTP server. Create, list, retrieve, delete, and admin metrics API routes.
Logic	Keyword-driven capability and risk detection, weighted score, effort estimates, dependency links, capacity-based sprint assignment.
Data	JSON plan records in data/plans.json. Created on first write; no external database required.

End-to-end flow

Feature input -> validation -> planning rules -> persistent plan -> history and admin metrics -> structured result. The JSON API is the integration point between browser, planner, and storage.

2. Frontend and planning logic

Inputs include product or team, feature idea, target audience, user reach, urgency, sprint length, and team point capacity. The server requires a description of at least 20 characters and limits it to 4,000 characters. The result is separated into product brief, priority assessment, user stories, tasks, sprint allocation, and risk notes. Plans can be reopened from history and exported as Markdown.

Priority score

Factor	Weight	Rule
Impact	40%	Customer outcome cues score 5; otherwise 3.
Reach	25%	User-selected rating from 2 to 5.
Urgency	20%	User-selected rating from 2 to 5.
Risk	15%	Risk keywords score 5; otherwise 2.

Score = weighted sum multiplied by 20, rounded to 0-100. Priority labels are P1 Now at 75+, P2 Next at 55-74, and P3 Later below 55. Factor values and weights appear in the result so users can inspect the decision.

Tasks and sprints

The planner starts with scope and design tasks, then detects up to four capability types from the idea. Tasks have story-point estimates, areas, and prerequisite IDs. Payment, integration, and file work use larger estimates; risk cues add a privacy review. Ordered tasks are packed into sequential sprints under a configurable capacity of 8-40 points and a sprint length of 1-4 weeks. All generated content is a draft for team review.

3. Data model and admin dashboard

Record group	Fields
Identity	UUID, title, product, creation timestamp.
Brief	Idea, audience, problem, objective, success signals.
Decision	Priority label and score, factor ratings, explanation, risk notes.
Delivery	Stories, task estimates, dependencies, sprint groups, capacity, duration.

Admin metrics

The dashboard reads total saved plans, average priority, total stories, estimated effort, and the six most recent plans. It supports deletion. It is a demo workspace view without authentication or role-based access controls. The JSON store is suitable for a classroom demonstration; production needs managed storage, authentication, authorization, backups, and concurrency-safe writes.

4. Git progression and launch

Six focused commits document the assignment stages: scaffold; planning engine; structured frontend; persistent records; admin monitoring; documentation and deployment notes. Local commit history is included in the project. A GitHub remote must be created and connected by the project owner.

Stage	Implementation evidence
1. Scaffold	Package metadata, project layout, run instructions.
2. Planner and API	Feature interpretation, scoring, task and sprint generation, API integration.
3. Interface	Structured inputs and formatted outputs.
4. Persistence	Save, list, retrieve, and delete plan records.
5. Admin	Aggregate metrics, recent activity, record management.
6. Documentation	System overview, launch notes, and demo walkthrough.

Local and Replit launch

Requires Node.js 18 or newer. Run npm start from the project folder and open http://localhost:3000. The server listens on process.env.PORT when set and binds to 0.0.0.0. Replit can run the same command. Persisted plan history requires writable persistent storage. No API keys or package installation are needed.

Suggested demo recording

1. Introduce Sprintwise and the feature planning problem. 2. Fill the example feature and change reach or urgency. 3. Generate a plan and show the score, stories, dependencies, estimates, and sprint groups. 4. Reopen the plan from history and export Markdown. 5. Open Admin overview to show workspace activity.

Delivery status

The runnable local project is included with a Replit launch note. The app has not been published to Replit or GitHub, and a hosted demo video has not been recorded. Deploy the project, create a GitHub remote, and record the suggested walkthrough to complete those external submission items.